

untangle::async

Generated by Doxygen 1.18.0

1	async	1
2	Topic Index	5
2.1	Topics	5
3	Namespace Index	7
3.1	Namespace List	7
4	Class Index	9
4.1	Class List	9
5	File Index	11
5.1	File List	11
6	Topic Documentation	13
6.1	namespace untangle: functions	13
7	Namespace Documentation	15
7.1	untangle Namespace Reference	15
7.1.1	Detailed Description	15
8	Class Documentation	17
8.1	untangle::async::attachment Struct Reference	17
8.1.1	Detailed Description	17
8.2	untangle::async::execution< actionT > Class Template Reference	17
8.2.1	Detailed Description	19
8.2.2	Constructor & Destructor Documentation	19
8.2.2.1	execution() [1/2]	19
8.2.2.2	execution() [2/2]	19
8.2.2.3	~execution()	20
8.2.3	Member Function Documentation	20
8.2.3.1	create_instance()	20
8.2.3.2	bind_action_and_method()	21
8.2.3.3	bind_action_and_function()	21
8.2.3.4	is_running()	22
8.2.3.5	is_busy()	22
8.2.3.6	run()	23
8.2.3.7	start()	23
8.2.3.8	stop()	23
8.2.3.9	add_action()	24
8.2.3.10	attach()	24
8.2.3.11	detach()	25
8.2.3.12	results()	25
8.2.4	Member Data Documentation	26
8.2.4.1	action_execute	26
8.2.4.2	action_stop	26

8.2.4.3 <code>action_is_running</code>	26
8.2.4.4 <code>on_finished</code>	26
8.2.4.5 <code>on_error</code>	27
8.3 <code>untangle::async::execution_poll</code> Class Reference	27
8.3.1 Detailed Description	28
8.3.2 Constructor & Destructor Documentation	28
8.3.2.1 <code>~execution_poll()</code>	28
8.3.3 Member Function Documentation	28
8.3.3.1 <code>add()</code>	28
8.3.3.2 <code>remove()</code>	28
8.3.3.3 <code>withdraw()</code>	29
8.3.3.4 <code>is_running()</code>	29
8.4 <code>untangle::async::invalid_attachment</code> Struct Reference	29
8.4.1 Detailed Description	30
8.4.2 Constructor & Destructor Documentation	30
8.4.2.1 <code>invalid_attachment()</code>	30
8.4.3 Member Function Documentation	30
8.4.3.1 <code>what()</code>	30
9 File Documentation	31
9.1 <code>async.hpp</code>	31
Index	39

Chapter 1

async

c++ mechanism to run queued callables in an asynchronous fashion

The interface consists of one header file that exposes a generic execution class. It is a wrapper of a queue of callables that are invoked on other thread. The callables are pumped into the queue by invoking an action from the caller thread.

The implementation is using the **actuator** callable, and therefore it exemplifies how a generic callable may improve the code structure, by easily creating interfaces inside a class instead of using external ones.

There are provided two execution modes: one-off and continuous.

requirements

C++23. The interface reports warnings with `std::println`, so it needs a standard library that provides `<print>` — GCC 14 or newer; older `libc++` and `MSVC` releases may not have it yet.

one-off execution



example: one-off execution of a plain function

```
#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer,
    // because an async binding keeps only weak ownership of it
    // the name is optional, and identifies the execution in the warnings it reports
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("oneoff");
```

```

std::function<void(void)> action;
// create async binding between the action and f; the execution is passed as a shared pointer,
// which the binding holds weakly - that is what lets the action outlive it safely
void_execution::bind_action_and_function(action, f, execution);
// whenever the action is invoked, a new callable wrapping f will be added to the execution's action queue
action();
// run the execution
execution->run();

// add the execution object to a poll of one's own; the poll answers for what was added to it
untangle::async::execution_poll poll;
poll.add(*execution);

// wait the polled executions to finish
while(poll.is_running())
{
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

return 0;
}

```

example: one-off execution of a class method

```

#include <async.hpp>

class A
{
public:
    void f(int x)
    {
        std::cout << "A::f " << x << " thread " << std::this_thread::get_id() << std::endl;
    }
    std::function<void(int)> action;
};

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    auto a = std::make_shared<A>();

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using int_execution = untangle::async::execution<std::function<void(int)>>;
    auto execution = int_execution::create_instance("oneoff_method");
    // note both the bound object and the execution must be shared pointers
    int_execution::bind_action_and_method(a->action, a, &A::f, execution);
    // whenever the action is invoked, a new callable wrapping f will be added to the execution's action queue
    a->action(10);
    // run the execution
    execution->run();

    // add the execution object to a poll of one's own; the poll answers for what was added to it
    untangle::async::execution_poll poll;
    poll.add(*execution);

    // wait the polled executions to finish
    while(poll.is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

continuous execution



example: continuous execution of a plain function

```

#include <async.hpp>

void f()
{

```

```

    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("continuous");
    std::function<void(void)> action;
    // create async binding between the action and f
    void_execution::bind_action_and_function(action, f, execution);

    // run the execution
    execution->start();
    // add f to the execution's queue (three times)
    action();action();action();
    // stop the execution
    execution->stop();

    // add the execution object to a poll of one's own; the poll answers for what was added to it
    untangle::async::execution_poll poll;
    poll.add(*execution);

    // wait the polled executions to finish
    while(poll.is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

queueing and lifetime

Queueing is thread safe. An action may be invoked from any thread; the execution's list is guarded, and the worker is woken rather than polled for.

`stop()` ends the execution's working life. The worker runs whatever was queued before the call and then leaves. An action invoked after `stop()` is refused rather than queued, because it would never run, and the refusal is reported:

```
warning: execution 'continuous' is stopped, action not added
```

An action bound to an object that has since been destroyed is dropped the same way, rather than ending the process:

```
warning: execution 'continuous' dropped an invalid action: bind: invalid object
```

The worker is detached and cannot be joined. Waiting for an execution therefore means waiting on its running state, which is what `execution_poll` reports and what the examples above do. The destructor waits on the same state, so an execution that goes out of scope while its worker is still running blocks until the worker is finished rather than leaving it reading freed memory.

The poll holds a pointer to each execution added to it, so an execution withdraws itself in its destructor - from every poll it was added to, since it may be in more than one. A poll destroyed first likewise releases what it still holds, so a poll may be a local. `execution_poll::remove()` is available for withdrawing one earlier; calling it is not required.

building the tests

```

cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build

```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access. The actuator submodule has to be present: `git submodule update --init`.

sanitizers

ASYNC_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DASYNC_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

The two answer different questions. ThreadSanitizer names both sides of a data race, which is how the queue's races were diagnosed. AddressSanitizer reports a use-after-free, which is how the dangling attachment was. Neither sees everything: a defect reached through a destroyed `std::mutex` throws `std::system_error` from inside `libsystem` before any instrumented load runs, so both sanitizers stay silent. Reach for a direct instrumented read when that happens.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [13](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::async::execution class	15
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::async::attachment	
One execution's place in the attachment graph: who attached it, if anyone	17
untangle::async::execution< actionT >	
Asynchronous template execution class	17
untangle::async::execution_poll	
Poll that reports whether any execution added to it is still running	27
untangle::async::invalid_attachment	
Raised by execution::attach() when the attachment asked for is refused	29

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

async.hpp		31
---------------------------	-------	----

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to [untangle::async::execution](#) class.

7.1.1 Detailed Description

Interface to [untangle::async::execution](#) class.

Chapter 8

Class Documentation

8.1 `untangle::async::attachment` Struct Reference

One execution's place in the attachment graph: who attached it, if anyone.

```
#include <async.hpp>
```

Public Attributes

- `std::weak_ptr< attachment >` `attacher`
The link of the execution that attached this one; empty when nothing has.

8.1.1 Detailed Description

One execution's place in the attachment graph: who attached it, if anyone.

At namespace scope, so it is one type rather than one per specialisation: an execution may be attached by an execution of any specialisation, and the chain is walked without knowing any link's action type. It dies with its execution, so it doubles as a liveness token - a link that still locks belongs to an execution that is still there.

The documentation for this struct was generated from the following file:

- `async.hpp`

8.2 `untangle::async::execution< actionT >` Class Template Reference

Asynchronous template execution class.

```
#include <async.hpp>
```

Public Member Functions

- `execution ()`
Constructs an execution with the default name. Call `create_instance()` instead.
- `execution (std::string exec_name)`
Constructs a named execution. Use `create_instance()` for one that is to be bound.
- `~execution ()`
Destroys the execution, once its worker has left.
- `bool is_running () const`
Is a worker of this execution's still there?
- `bool is_busy () const`
Is this execution working through its action queue?
- `void run ()`
Runs what is queued on a worker of its own, once, and keeps the results.
- `void start ()`
Starts a continuous worker that runs whatever is queued until `stop()`.
- `void stop ()`
Stops the worker, after the actions already queued have run.
- `template<typename... Args>`
`bool add_action (actionT action, Args &&... args)`
Queues an action, bound to args, and says whether it was taken.
- `template<typename otherT>`
`void attach (const std::shared_ptr< otherT > &other)`
Attaches another execution, so that this one triggers it.
- `template<typename otherT>`
`bool detach (otherT &other)`
Detaches an execution previously attached by `attach()`.
- `auto results () const`
The return values of the actions run by `run()`, in the order they ran.

Static Public Member Functions

- `template<typename... Args>`
`static std::shared_ptr< execution > create_instance (Args &&... args)`
Creates an execution owned by a `std::shared_ptr`, which is what binding to it requires.
- `template<typename classT, typename T>`
`static void bind_action_and_method (actionT &action, const std::shared_ptr< classT > &obj, T classT::*method, const std::shared_ptr< execution > &async_exec)`
Binds asynchronously an external action to a class function member.
- `template<typename T>`
`static void bind_action_and_function (actionT &action, T Fn, const std::shared_ptr< execution > &async_exec)`
Binds asynchronously an external action to a plain function.

Public Attributes

- `std::function< void(void)> action_execute`
Connection point that runs this execution's queued actions. Bound to `execute_actions()`.
- `std::function< void(void)> action_stop`
Connection point that stops this execution. Bound to `stop()`.
- `std::function< bool(void)> action_is_running`
Connection point that reports whether this execution is running. Bound to `is_running()`.
- `std::function< void(void)> on_finished`
Called on the worker's thread once the action queue has drained. Assigned by the caller.
- `std::function< void(std::exception_ptr)> on_error`
Called on the worker's thread with whatever an action threw. Assigned by the caller.

Static Public Attributes

- `static constexpr auto default_name = "default_name"`
Name of an execution built unnamed.

8.2.1 Detailed Description

```
template<typename actionT>
class untangle::async::execution< actionT >
```

Asynchronous template execution class.

An async execution provides a mechanism to queue actions and execute them sequentially on a separate thread. It may also attach another [execution](#) object and trigger its actions. This way actions of different types may be executed on the same thread.

The mechanism relies on an "asynchronous binding" created by [bind_action_and_method\(\)](#) or [bind_action_and_function\(\)](#).

Template Parameters

<code>actionT</code>	It represents the type of the action. It is specified as <code>std::function<...></code> and should match the signature of the bound function or class method.
----------------------	--

8.2.2 Constructor & Destructor Documentation

8.2.2.1 `execution()` [1/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution () [inline]
```

Constructs an execution with the default name. Call [create_instance\(\)](#) instead.

The name identifies this execution in the warnings it reports, and is otherwise unused.

8.2.2.2 `execution()` [2/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution (
    std::string exec_name) [inline], [explicit]
```

Constructs a named execution. Use [create_instance\(\)](#) for one that is to be bound.

Parameters

exec_name	- A name for this execution.
-----------	------------------------------

8.2.2.3 ~execution()

```
template<typename actionT>
untangle::async::execution< actionT >::~~execution () [inline]
```

Destroys the execution, once its worker has left.

The worker is detached and cannot be joined, so this waits on the "running" state instead. It detaches this execution from whatever attached it first, and returns at once for one that was never started.

Attention

Safe only because running_ is cleared as the very last thing the worker does: this object may be freed the moment it reads false, so nothing may follow it in execute() or loop().

Remarks

No rule of five, deliberately. An execution is declared once and used in place; the members already delete the copy operations several times over.

8.2.3 Member Function Documentation

8.2.3.1 create_instance()

```
template<typename actionT>
template<typename... Args>
std::shared_ptr< execution > untangle::async::execution< actionT >::create_instance (
    Args &&... args) [inline], [static]
```

Creates an execution owned by a std::shared_ptr, which is what binding to it requires.

An execution built any other way is still usable - [add_action\(\)](#) takes actions directly - but cannot be bound, and the attempt does not compile.

Parameters

args	- Constructor arguments: a name, or nothing for the default name.
------	---

Returns

- A std::shared_ptr owning the new execution.

8.2.3.2 bind_action_and_method()

```
template<typename actionT>
template<typename classT, typename T>
void untangle::async::execution< actionT >::bind_action_and_method (
    actionT & action,
    const std::shared_ptr< classT > & obj,
    T classT::* method,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a class function member.

It creates an action as a binding to a class method (by untangle::bind()), and assigns to action a callable that queues it. The execution is held weakly, so the action may outlive it; the action itself is shared with every call queued from it, rather than copied per call.

Attention

Invoking action after the execution has been destroyed throws untangle::invalid_action. The worker catches it and drops the action with a warning.

Parameters

action	[in,out] - An action of type std::function<...>.
obj	- A std::shared_ptr that wraps the bound class object.
method	- Pointer to function member. It is specified as &<class type>::<function member>.
async_exec	- A std::shared_ptr owning the execution the action is queued on. Requiring it here is what keeps an execution that no shared_ptr owns from being bound at all.

8.2.3.3 bind_action_and_function()

```
template<typename actionT>
template<typename T>
void untangle::async::execution< actionT >::bind_action_and_function (
    actionT & action,
    T Fn,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a plain function.

It wraps Fn in an action and assigns to action a callable that queues it. The execution is held weakly, as in [bind_action_and_method\(\)](#), and the action is shared with every call queued from it rather than copied per call.

Parameters

action	[in,out] - An action of type std::function<...>.
Fn	- A plain function, or any callable; moved into the binding.

async_exec	- A std::shared_ptr owning the execution the action is queued on.
------------	---

8.2.3.4 is_running()

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_running () const [inline]
```

Is a worker of this execution's still there?

True from [run\(\)](#) or [start\(\)](#) until the worker has left, which is the same instant [~execution\(\)](#) waits for and the same one [run\(\)](#) waits for before starting another.

Remarks

[on_finished](#) is raised from inside the drain and reads this as true - the worker is standing in the callback with the rest of its path still to go.

Returns

true - a worker is still there.

8.2.3.5 is_busy()

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_busy () const [inline]
```

Is this execution working through its action queue?

True from the moment an action is added until the last one has returned. Unlike [is_running\(\)](#), which is about the worker thread, this answers "has it room for more work"

- a worker started by [start\(\)](#) is running for its whole life whether busy or idle.

Remarks

Answers for this execution's own actions, not for those it triggers through [attach\(\)](#).

A false answer is durable only for the caller that is itself adding the actions; a true one may stop being true the moment after.

Returns

true - actions are queued, or one is running.

8.2.3.6 run()

```
template<typename actionT>
void untangle::async::execution< actionT >::run () [inline]
```

Runs what is queued on a worker of its own, once, and keeps the results.

The worker is detached: wait for it with `is_running()` or `execution_poll`. Actions queued while it is draining are run by it too. `results()` is filled by this path only.

Blocks until a worker still resident from a previous run has left, so one execution never has two. `is_running()` reads false at exactly that moment, so a caller that waits for it and then runs again does not wait here at all.

Attention

Never call this from inside an action: the worker that would have to leave is the one making the call, so it waits for itself and hangs.

8.2.3.7 start()

```
template<typename actionT>
void untangle::async::execution< actionT >::start () [inline]
```

Starts a continuous worker that runs whatever is queued until `stop()`.

Unlike `run()`, the worker stays for the life of the execution, so `is_running()` is true whether or not there is work; ask `is_busy()` instead. Results are not collected.

Blocks until a worker still resident has left, as `run()` does.

Attention

Never call this from inside an action, for the reason given on `run()`.

8.2.3.8 stop()

```
template<typename actionT>
void untangle::async::execution< actionT >::stop () [inline]
```

Stops the worker, after the actions already queued have run.

Remarks

An action passed to `add_action()` after this call is refused, not queued: `stop()` is the end of this execution's working life, and an action accepted then would never run.

8.2.3.9 add_action()

```
template<typename actionT>
template<typename... Args>
bool untangle::async::execution< actionT >::add_action (
    actionT action,
    Args &&... args) [inline]
```

Queues an action, bound to args, and says whether it was taken.

It runs later, on whichever thread drives this execution - its own worker, or an attacher's. Queueing before [run\(\)](#) or [start\(\)](#) is normal.

Attention

A stopped execution refuses and drops the action, silently for a caller that ignores the answer. Nothing is thrown: `untangle::invalid_action` means a dead binding, while a refused action is sound and merely too late.

Remarks

A caller arriving through [bind_action_and_method\(\)](#) or [bind_action_and_function\(\)](#) never sees the answer: those lambdas return `actionT::result_type`, which has no room for it.

Template Parameters

Args	- The argument types the action is bound to.
------	--

Parameters

action	- The action to queue.
args	- The arguments to bind to action.

Returns

true - queued, and this execution's worker will run it.
false - the execution is stopped and the action was dropped.

8.2.3.10 attach()

```
template<typename actionT>
template<typename otherT>
void untangle::async::execution< actionT >::attach (
    const std::shared_ptr< otherT > & other) [inline]
```

Attaches another execution, so that this one triggers it.

The attached execution's pending actions are run by this one's worker, and stopping this one stops that one too. Actions of different types can be run on one thread this way.

Parameters

other	- A std::shared_ptr owning the execution to trigger; requiring one keeps an unowned execution from being attached at all. Attaching the same execution twice adds it twice, and one detach() still removes it completely.
-------	---

Exceptions

invalid_attachment	- if other is attached already. An execution has at most one attacher, which keeps the graph a forest and the cycle check a walk rather than a search.
invalid_attachment	- if the attachment would close a cycle, itself included. Driving a cycle recurses until the stack is gone, so it is refused where the caller can still be told.

8.2.3.11 detach()

```
template<typename actionT>
template<typename otherT>
bool untangle::async::execution< actionT >::detach (
    otherT & other) [inline]
```

Detaches an execution previously attached by attach().

Both paths are unwired together, the triggering one and the stopping one.

Parameters

other	- The execution to stop triggering. Detaching one that was never attached is not an error: the caller asked for a state that already holds.
-------	---

Returns

true - other was attached and is no longer.
false - other was not attached to this execution.

8.2.3.12 results()

```
template<typename actionT>
auto untangle::async::execution< actionT >::results () const [inline]
```

The return values of the actions run by run(), in the order they ran.

Read it from on_finished, which fires once the queue has drained, so the results are complete by the time the callback sees them. Returned by value: the worker is appending to it.

Remarks

Only run() fills this. A continuous worker has no point at which a run is over, so start() collects nothing.

Returns

The results of the most recent run, or empty if none has produced any.

8.2.4 Member Data Documentation

8.2.4.1 `action_execute`

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_execute
```

Connection point that runs this execution's queued actions. Bound to `execute_actions()`.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.2 `action_stop`

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_stop
```

Connection point that stops this execution. Bound to `stop()`.

`attach()` wires it together with `action_execute`, and `detach()` unwires both.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.3 `action_is_running`

```
template<typename actionT>
std::function<bool(void)> untangle::async::execution< actionT >::action_is_running
```

Connection point that reports whether this execution is running. Bound to `is_running()`.

`execution_poll` holds its address, and `~execution()` takes it back out.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.4 `on_finished`

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::on_finished
```

Called on the worker's thread once the action queue has drained. Assigned by the caller.

`results()` is complete by the time it runs, and `is_running()` reads true there.

8.2.4.5 on_error

```
template<typename actionT>
std::function<void(std::exception_ptr)> untangle::async::execution< actionT >::on_error
```

Called on the worker's thread with whatever an action threw. Assigned by the caller.

The action is dropped either way; this is what it is dropped to. Without it the only record is a warning on stderr naming this execution, which reaches a log and no code — an action that failed and one that succeeded are the same from outside. A handler assigned here replaces that warning, so a caller that takes responsibility for reporting is not reported twice.

Remarks

It receives a `std::exception_ptr` because the worker catches everything, including what is not a `std::exception`. Rethrow it to read it.

It is told about a dead binding too, which arrives as `untangle::invalid_action`. A caller asking why an action did not run wants both answers, not the one the header happens to raise itself.

Attention

It runs on the worker's thread, inside the drain, like `on_finished`. Nothing may escape it: it is called from the same try the action was called from, and an exception leaving a detached thread function calls `std::terminate`.

The documentation for this class was generated from the following file:

- `async.hpp`

8.3 untangle::async::execution_poll Class Reference

Poll that reports whether any `execution` added to it is still running.

```
#include <async.hpp>
```

Public Member Functions

- `execution_poll ()=default`
Constructs an empty poll. `add()` is what puts anything in it.
- `~execution_poll ()`
Destroys the poll, releasing every execution still in it.
- `template<typename asyncexecT>`
`void add (asyncexecT &async_exec)`
Adds an `execution` object to the poll.
- `template<typename asyncexecT>`
`void remove (asyncexecT &async_exec)`
Removes an `execution` object from the poll.
- `void withdraw (const std::function< bool(void)> *action)`
Takes one action pointer out, without touching the execution it came from.
- `auto is_running ()`
Checks if the execution objects are running.

8.3.1 Detailed Description

Poll that reports whether any [execution](#) added to it is still running.

Remarks

An execution runs in a detached thread and cannot be joined, so checking the "running" state is the only way to wait for one. This checks any number of them at once.

A poll is constructed and owned by the caller, and answers for the executions added to that one. An owner of a set of workers - a thread pool is the case this was written for - waits for its own and not for whatever else the process is running.

8.3.2 Constructor & Destructor Documentation

8.3.2.1 ~execution_poll()

```
untangle::async::execution_poll::~execution_poll () [inline]
```

Destroys the poll, releasing every execution still in it.

Remarks

The record runs both ways, and it has to: each side holds a pointer into the other, so whichever dies first takes itself out of the other. ~execution() withdraws from every poll holding it; this tells every execution it holds that this poll is gone. Neither order leaves a dangling pointer, which is what lets a poll be a local.

8.3.3 Member Function Documentation

8.3.3.1 add()

```
template<typename asyncexecT>
void untangle::async::execution_poll::add (
    asyncexecT & async_exec) [inline]
```

Adds an [execution](#) object to the poll.

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.3.2 remove()

```
template<typename asyncexecT>
void untangle::async::execution_poll::remove (
    asyncexecT & async_exec) [inline]
```

Removes an [execution](#) object from the poll.

Attention

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.3.3 withdraw()

```
void untangle::async::execution_poll::withdraw (
    const std::function< bool(void)> * action) [inline]
```

Takes one action pointer out, without touching the execution it came from.

What ~execution() calls for each poll holding it: by then the object is being destroyed and must not be called back into, so the two halves of [remove\(\)](#) are separated here.

Parameters

action	- The address add() stored.
--------	---

8.3.3.4 is_running()

```
auto untangle::async::execution_poll::is_running () [inline]
```

Checks if the execution objects are running.

Returns

true - if at least one [execution](#) object in this poll is running.

The documentation for this class was generated from the following file:

- [async.hpp](#)

8.4 untangle::async::invalid_attachment Struct Reference

Raised by [execution::attach\(\)](#) when the attachment asked for is refused.

```
#include <async.hpp>
```

Public Member Functions

- [invalid_attachment](#) (std::string text)
Construct a new invalid attachment object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

8.4.1 Detailed Description

Raised by `execution::attach()` when the attachment asked for is refused.

Remarks

Not an `untangle::invalid_action`: that one reports a binding whose target has died and is swallowed by the worker, while this is a caller error and must not be.

8.4.2 Constructor & Destructor Documentation

8.4.2.1 `invalid_attachment()`

```
untangle::async::invalid_attachment::invalid_attachment (
    std::string text) [inline], [explicit]
```

Construct a new invalid attachment object.

Parameters

text	- A message text, describing the reason of this exception.
------	--

8.4.3 Member Function Documentation

8.4.3.1 `what()`

```
const char * untangle::async::invalid_attachment::what () const [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

`const char*` - The message text.

The documentation for this struct was generated from the following file:

- `async.hpp`

Chapter 9

File Documentation

9.1 async.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <actuator/actuator.hpp>
00009 #include <algorithm>
00010 #include <atomic>
00011 #include <chrono>
00012 #include <condition_variable>
00013 #include <exception>
00014 #include <functional>
00015 #include <memory>
00016 #include <mutex>
00017 #include <print>
00018 #include <string>
00019 #include <thread>
00020 #include <type_traits>
00021 #include <utility>
00022 #include <vector>
00023
00024 namespace untangle {
00025 namespace async {
00026
00036 struct invalid_attachment : std::exception {
00042     explicit invalid_attachment(std::string text) : message_(std::move(text)) {}
00043
00049     const char* what() const noexcept override { return message_.c_str(); }
00050
00051 private:
00052     std::string message_;
00053 };
00054
00063 struct attachment {
00065     std::weak_ptr<attachment> attacher;
00066 };
00067
00078 class execution_poll {
00079 public:
00083     execution_poll() = default;
00084
00093     ~execution_poll() {
00094         std::vector<registration> held;
00095
00096         {
00097             std::lock_guard<std::mutex> lock(actuator_mutex_);
00098             held.swap(registrations_);
00099         }
00100
00101         // Outside the lock: each of these takes the execution's own, and ~execution() takes them in
00102         // the opposite order. Releasing this one first is what keeps the two from meeting in a cycle.
00103         for (auto& one : held) {
00104             one.forget();
00105         }
00106     }
00107
00113 template <typename asyncexecT>
00114 void add(asyncexecT& async_exec) {
```

```

00115 {
00116     std::lock_guard<std::mutex> lock(actuator_mutex_);
00117
00118     if (!actuator_is_running_.is_connected()) {
00119         actuator_is_running_ = untangle::connect(async_exec.action_is_running);
00120     } else {
00121         actuator_is_running_.add(&async_exec.action_is_running);
00122     }
00123
00124     registrations_.emplace_back(&async_exec.action_is_running,
00125                                 [this, &async_exec] { async_exec.forget_poll(*this); });
00126 }
00127
00128 // Outside the lock, and deliberately: this takes the execution's own. The execution has to
00129 // know which polls hold a pointer into it, because ~execution() is what takes it back out and
00130 // there is now more than one poll it could be in.
00131 async_exec.remember_poll(*this);
00132 }
00133
00142 template <typename asyncexecT>
00143 void remove(asyncexecT& async_exec) {
00144     withdraw(&async_exec.action_is_running);
00145     async_exec.forget_poll(*this);
00146 }
00147
00156 void withdraw(const std::function<bool(void)>* action) {
00157     std::lock_guard<std::mutex> lock(actuator_mutex_);
00158
00159     actuator_is_running_.remove(action);
00160     std::erase_if(registrations_,
00161                  [action](const registration& one) { return one.action == action; });
00162 }
00163
00169 auto is_running() {
00170     std::lock_guard<std::mutex> lock(actuator_mutex_);
00171     actuator_is_running_();
00172
00173     auto result = false;
00174     for (const auto& ret : actuator_is_running_.results) {
00175         result = result || ret;
00176     }
00177     return result;
00178 }
00179
00180 private:
00185 struct registration {
00186     const std::function<bool(void)>* action;
00187     std::function<void()> forget;
00188 };
00189
00190 actuator<std::function<bool(void)>> actuator_is_running_;
00191 std::vector<registration> registrations_;
00192 mutable std::mutex actuator_mutex_;
00193 };
00194
00208 template <typename actionT>
00209 class execution {
00210     // An execution may attach one of a different specialisation, which is a different class;
00211     // attach() and detach() reach into its attachment_lifetime_.
00212     template <typename otherActionT>
00213     friend class execution;
00214
00215     // add() and remove() keep the record this object holds of the polls that hold it.
00216     friend class execution_poll;
00217
00218 public:
00228     template <typename... Args>
00229     static std::shared_ptr<execution> create_instance(Args&&... args) {
00230         return std::make_shared<execution>(std::forward<Args>(args)...);
00231     }
00232
00238     execution() : execution(std::string(default_name)) {}
00239
00245     explicit execution(std::string exec_name) : name(std::move(exec_name)) {
00246         action_execute = untangle::bind(this, &execution<actionT>::execute_actions);
00247         action_stop = untangle::bind(this, &execution<actionT>::stop);
00248         action_is_running = untangle::bind(this, &execution<actionT>::is_running);
00249     }
00250
00265     ~execution() {
00266         // Out of the attacher first, before the worker is even asked to stop: from here on nothing
00267         // may reach this object, and the actions about to be destroyed are the ones it holds.
00268         if (!attachment_lifetime_ -> attacher.expired()) {
00269             attacher_execute -> remove(&action_execute);
00270             attacher_stop -> remove(&action_stop);
00271         }
00272

```

```

00273 {
00274     std::lock_guard<std::mutex> lock(action_mutex_);
00275     started_ = false;
00276     stopped_ = true;
00277 }
00278
00279 action_cv_.notify_all();
00280
00281 while (running_.load()) { // time of check
00282     std::this_thread::sleep_for(std::chrono::milliseconds(1));
00283 }
00284
00285 // time of use: this object is freed once the destructor returns, so the check above is only
00286 // safe because running_ is the last thing the worker touches.
00287 //
00288 // Out of every poll that holds a pointer into this object: an execution can be in several,
00289 // and an address left in any of them outlives the object. Taken by swap so nothing else can be
00290 // added to the record while this walks it, and withdrawn rather than removed because remove()
00291 // would call back into an object that is going away.
00292 std::vector<execution_poll*> holding_polls;
00293
00294 {
00295     std::lock_guard<std::mutex> lock(polls_mutex_);
00296     holding_polls.swap(polls_);
00297 }
00298
00299 for (auto* poll : holding_polls) {
00300     poll->withdraw(&action_is_running);
00301 }
00302 }
00303
00321 template <typename classT, typename T>
00322 static void bind_action_and_method(actionT& action, const std::shared_ptr<classT>& obj,
00323                                     T classT::* method,
00324                                     const std::shared_ptr<execution>& async_exec) {
00325     auto async_action = std::make_shared<const actionT>(untangle::bind(obj, method));
00326
00327     action = [wp = std::weak_ptr<execution>(async_exec),
00328              async_action](auto&&... args) -> actionT::result_type {
00329         // lock() also keeps the execution alive for the duration of the call
00330         const auto exec = wp.lock();
00331         if (!exec) {
00332             throw invalid_action("bind: invalid execution");
00333         }
00334
00335         exec->add_queued_action([async_action, ... args = std::forward<decltype(args)>(args)] {
00336             return (*async_action)(args...);
00337         });
00338
00339         return typename actionT::result_type();
00340     };
00341 }
00342
00354 template <typename T>
00355 static void bind_action_and_function(actionT& action, T Fn,
00356                                     const std::shared_ptr<execution>& async_exec) {
00357     auto async_action = std::make_shared<const actionT>(std::move(Fn));
00358
00359     action = [wp = std::weak_ptr<execution>(async_exec),
00360              async_action](auto&&... args) -> actionT::result_type {
00361         const auto exec = wp.lock();
00362         if (!exec) {
00363             throw invalid_action("bind: invalid execution");
00364         }
00365
00366         exec->add_queued_action([async_action, ... args = std::forward<decltype(args)>(args)] {
00367             return (*async_action)(args...);
00368         });
00369
00370         return typename actionT::result_type();
00371     };
00372 }
00373
00385 bool is_running() const { return running_.load(); }
00386
00402 bool is_busy() const {
00403     std::lock_guard<std::mutex> lock(action_mutex_);
00404     return has_pending_actions() || executing_action_.load();
00405 }
00406
00420 void run() {
00421     wait_thread_to_finish();
00422     collecting_results_ = true;
00423     thread_ = std::thread(&execution::execute, this);
00424     thread_.detach();
00425 }
00426

```

```

00437 void start() {
00438     wait_thread_to_finish();
00439     collecting_results_ = false;
00440     {
00441         std::lock_guard<std::mutex> lock(action_mutex_);
00442         started_ = true;
00443         stopped_ = false;
00444     }
00445
00446     thread_ = std::thread(&execution::loop, this);
00447     thread_.detach();
00448 }
00449
00450 void stop() {
00451     {
00452         std::lock_guard<std::mutex> lock(action_mutex_);
00453         started_ = false;
00454         stopped_ = true;
00455     }
00456
00457     // The worker drains what is already queued before it leaves loop(), so stop() does not have to
00458     // spin on the queue; waking it is enough.
00459     action_cv_.notify_all();
00460
00461     if (actuator_stop_.is_connected()) {
00462         actuator_stop_();
00463     }
00464 }
00465
00466 template <typename... Args>
00467 bool add_action(actionT action, Args&&... args) {
00468     return add_queued_action(std::bind(std::move(action), std::forward<Args>(args)...));
00469 }
00470
00471 template <typename otherT>
00472 void attach(const std::shared_ptr<otherT>& other) {
00473     if (static_cast<const void*>(other.get()) == static_cast<const void*>(this)) {
00474         throw invalid_attachment("attach: an execution cannot be attached to itself");
00475     }
00476
00477     if (!other->attachment_lifetime_->attacher.expired()) {
00478         throw invalid_attachment("attach: that execution is attached already");
00479     }
00480
00481     // At most one attacher each makes the graph a forest, so a cycle closes exactly when other is
00482     // somewhere up this chain. Walking it is the whole check.
00483     for (auto link = attachment_lifetime_->attacher.lock(); link; link = link->attacher.lock()) {
00484         if (link == other->attachment_lifetime_) {
00485             throw invalid_attachment("attach: the attachment would close a cycle");
00486         }
00487     }
00488
00489     if (!actuator_execute_.is_connected()) {
00490         actuator_execute_ = untangle::connect(other->action_execute);
00491     } else {
00492         actuator_execute_.add(&other->action_execute);
00493     }
00494
00495     if (!actuator_stop_.is_connected()) {
00496         actuator_stop_ = untangle::connect(other->action_stop);
00497     } else {
00498         actuator_stop_.add(&other->action_stop);
00499     }
00500
00501     // What ~execution() needs to take itself back out of these actuators: the link proves this
00502     // attacher is still there, and the two pointers are only ever followed while it does.
00503     other->attachment_lifetime_->attacher = attachment_lifetime_;
00504     other->attacher_execute_ = &actuator_execute_;
00505     other->attacher_stop_ = &actuator_stop_;
00506 }
00507
00508 template <typename otherT>
00509 bool detach(otherT& other) {
00510     // The actuator's own action list is the record of what is attached; nothing else has to keep
00511     // one. An action is stored as a pointer, so the attachment is found by identity.
00512     const auto& actions = actuator_execute_.actions;
00513     if (std::find(actions.begin(), actions.end(), &other.action_execute) == actions.end()) {
00514         return false;
00515     }
00516
00517     actuator_execute_.remove(&other.action_execute);
00518     actuator_stop_.remove(&other.action_stop);
00519
00520     other.attachment_lifetime_->attacher.reset();
00521     other.attacher_execute_ = nullptr;
00522     other.attacher_stop_ = nullptr;
00523 }

```

```

00581     return true;
00582 }
00583
00595 auto results() const {
00596     std::lock_guard<std::mutex> lock(results_mutex_);
00597     return results_;
00598 }
00599
00600 static constexpr auto default_name = "default_name";
00601
00607 std::function<void(void)> action_execute;
00608
00616 std::function<void(void)> action_stop;
00617
00626 std::function<bool(void)> action_is_running;
00627
00633 std::function<void(void)> on_finished;
00634
00654 std::function<void(std::exception_ptr)> on_error;
00655
00656 std::string name = default_name;
00657
00658 private:
00659 using queued_action_t = std::function<typename actionT::result_type(void)>;
00660
00665 void remember_poll(execution_poll& poll) {
00666     std::lock_guard<std::mutex> lock(polls_mutex_);
00667
00668     polls_.push_back(&poll);
00669 }
00670
00674 void forget_poll(execution_poll& poll) {
00675     std::lock_guard<std::mutex> lock(polls_mutex_);
00676
00677     std::erase(polls_, &poll);
00678 }
00679
00687 bool has_pending_actions() const { return action_actuator_.is_connected(); }
00688
00698 bool add_queued_action(queued_action_t action) {
00699     {
00700         std::lock_guard<std::mutex> lock(action_mutex_);
00701
00702         // Once stopped, the worker is on its way out and would never reach this action; dropping it
00703         // here is what keeps it from sitting in the queue looking as though it were pending.
00704         if (stopped_) {
00705             std::println(stderr, "warning: execution '{}' is stopped, action not added", name);
00706             return false;
00707         }
00708
00709         action_actuator_.add(std::move(action));
00710     }
00711
00712     action_cv_.notify_one();
00713     return true;
00714 }
00715
00722 void execute_action(queued_action_t& action) {
00723     if constexpr (std::is_void_v<typename actionT::result_type>) {
00724         action();
00725     } else {
00726         auto value = action();
00727
00728         if (collecting_results_) {
00729             std::lock_guard<std::mutex> lock(results_mutex_);
00730             results_.push_back(std::move(value));
00731         }
00732     }
00733 }
00734
00744 void execute_actions() {
00745     actions_run_ = 0;
00746
00747     for (;;) {
00748         // The whole batch is taken out under the lock and run with the lock released: an action is
00749         // caller code that may run for a while, and may itself call add_action(), which takes the
00750         // same mutex. Moving it out is also what keeps action_actuator_ to one thread - it is not
00751         // synchronised, so a producer adding to it must never be writing a list the worker is
00752         // walking. A moved-from actuator is empty and its handles stay valid, which is what
00753         // actuator_test's test_move_carries_the_owned_actions_and_empties_the_source states.
00754         actuator<queued_action_t> batch;
00755
00756         {
00757             std::lock_guard<std::mutex> lock(action_mutex_);
00758             if (!has_pending_actions()) {
00759                 break;
00760             }

```

```

00761
00762     batch = std::move(action_actuator_);
00763
00764     // Under the lock that emptied the pending actions, so is_busy() sees the two together or
00765     // neither.
00766     executing_action_ = true;
00767 }
00768
00769 for (auto* action : batch.actions) {
00770     // Nothing may leave this loop: an exception escaping a detached thread function calls
00771     // std::terminate, and one escaping mid-batch would take the actions behind it with it.
00772     try {
00773         execute_action(*action);
00774     } catch (const invalid_action& ia) {
00775         if (!report_error()) {
00776             std::println(stderr, "warning: execution '{}' dropped an invalid action: {}", name,
00777                 ia.what());
00778         }
00779     } catch (const std::exception& e) {
00780         if (!report_error()) {
00781             std::println(stderr, "warning: execution '{}' dropped an action that threw: {}", name,
00782                 e.what());
00783         }
00784     } catch (...) {
00785         if (!report_error()) {
00786             std::println(stderr,
00787                 "warning: execution '{}' dropped an action that threw an unknown type",
00788                 name);
00789         }
00790     }
00791
00792     // Counted whether it ran cleanly, reported a dead binding, or threw: it was in the batch,
00793     // and the batch is what the notification is about.
00794     ++actions_run_;
00795 }
00796
00797 executing_action_ = false;
00798
00799 // The notification is raised without the lock: on_finished may call add_action(), which
00800 // takes it.
00801 bool drained = false;
00802 {
00803     std::lock_guard<std::mutex> lock(action_mutex_);
00804     drained = !has_pending_actions();
00805 }
00806
00807 if (drained) {
00808     notify_finished();
00809 }
00810 }
00811
00812 // Last, and part of the pass: an attachment is only ever reached through action_execute, which
00813 // is this function, so whoever drives this execution drives the ones attached to it.
00814 if (actuator_execute_.is_connected()) {
00815     actuator_execute_();
00816 }
00817 }
00818
00819 bool report_error() {
00820     if (!on_error) {
00821         return false;
00822     }
00823 }
00824
00825 // The handler is the caller's code, running on a detached thread inside the try that caught the
00826 // action. Anything escaping it would leave the thread function and call std::terminate, so it
00827 // is caught here - a caller's broken handler costs its own warning, not the process.
00828 try {
00829     on_error(std::current_exception());
00830 } catch (...) {
00831     std::println(stderr, "warning: execution '{}' had on_error throw, and dropped what it threw",
00832         name);
00833 }
00834
00835 return true;
00836 }
00837
00838 void notify_finished() {
00839     if (actions_run_ == 0 || !on_finished) {
00840         return;
00841     }
00842 }
00843
00844 {
00845     // An action may queue another, so a pass that drained can leave more behind it. That is the
00846     // next batch, not the end of this one.
00847     std::lock_guard<std::mutex> lock(action_mutex_);
00848     if (has_pending_actions()) {
00849         return;
00850     }
00851 }

```

```

00861     }
00862 }
00863
00864 on_finished();
00865 }
00866
00874 void wait_thread_to_finish() {
00875     auto resident = false;
00876     while (!running_.compare_exchange_weak(resident, true)) {
00877         resident = false;
00878         std::this_thread::sleep_for(std::chrono::milliseconds(1));
00879     }
00880 }
00881
00882 void execute() {
00883     // The results belong to this run, not to this run and the last one.
00884     {
00885         std::lock_guard<std::mutex> lock(results_mutex_);
00886         results_.clear();
00887     }
00888
00889     execute_actions();
00890
00891     std::println("execution '{}'' finishing thread", name);
00892
00893     // Must stay last: ~execution() may free this object the moment it reads false.
00894     running_ = false;
00895 }
00896
00897 void loop() {
00898     for (;;) {
00899         {
00900             std::unique_lock<std::mutex> lock(action_mutex_);
00901
00902             // Bounded: an attached execution has its own list and cannot notify this condition
00903             // variable.
00904             action_cv_.wait_for(lock, std::chrono::milliseconds(10),
00905                 [this] { return has_pending_actions() || !started_; });
00906
00907             if (!started_ && !has_pending_actions()) {
00908                 break;
00909             }
00910         }
00911         execute_actions();
00912     }
00913
00914     // A last pass for the attachments; this execution's own list is already empty here.
00915     execute_actions();
00916
00917     std::println("execution '{}'' thread finished", name);
00918
00919     // Must stay last: ~execution() may free this object the moment it reads false.
00920     running_ = false;
00921 }
00922
00923 std::thread thread_;
00924
00925 std::atomic_bool started_ = {false};
00926 std::atomic_bool running_ = {false};
00927
00928 // Set by stop(), and the difference between "not started yet" and "finished for good": actions
00929 // may be queued before run()/start(), but not after stop().
00930 std::atomic_bool stopped_ = {false};
00931
00932 // action_actuator_, started_ and stopped_ are written by every thread that calls add_action() or
00933 // stop() and read by the worker; nothing touches them outside this mutex.
00934 mutable std::mutex action_mutex_;
00935 std::condition_variable action_cv_;
00936
00945 actuator<queued_action_t> action_actuator_;
00946
00954 using void_actuator = actuator<std::function<void(void)>>;
00955
00956 void_actuator actuator_execute_;
00957 void_actuator actuator_stop_;
00958
00970 std::shared_ptr<attachment> attachment_lifetime_ = std::make_shared<attachment>();
00971
00983 void_actuator* attacher_execute_ = nullptr;
00984 void_actuator* attacher_stop_ = nullptr;
00985
00986 // std::vector cannot hold void type; use an arbitrary type e.g. int
00987 using resultT = std::conditional<std::is_void<typename actionT::result_type>::value, int,
00988     typename actionT::result_type>;
00989
00993 std::vector<typename resultT::type> results_;
00994

```

```
00998 mutable std::mutex results_mutex_;
00999
01006 std::atomic_bool collecting_results_ = {false};
01007
01015 std::atomic_bool executing_action_ = {false};
01016
01018 std::vector<execution_poll*> polls_;
01019 mutable std::mutex polls_mutex_;
01020
01028 std::atomic_size_t actions_run_ = {0};
01029 };
01030 } // namespace async
01031 } // namespace untangle
```

Index

- ~execution
 - untangle::async::execution< actionT >, [20](#)
- ~execution_poll
 - untangle::async::execution_poll, [28](#)
- action_execute
 - untangle::async::execution< actionT >, [26](#)
- action_is_running
 - untangle::async::execution< actionT >, [26](#)
- action_stop
 - untangle::async::execution< actionT >, [26](#)
- add
 - untangle::async::execution_poll, [28](#)
- add_action
 - untangle::async::execution< actionT >, [23](#)
- async, [1](#)
- attach
 - untangle::async::execution< actionT >, [24](#)
- bind_action_and_function
 - untangle::async::execution< actionT >, [21](#)
- bind_action_and_method
 - untangle::async::execution< actionT >, [20](#)
- create_instance
 - untangle::async::execution< actionT >, [20](#)
- detach
 - untangle::async::execution< actionT >, [25](#)
- execution
 - untangle::async::execution< actionT >, [19](#)
- invalid_attachment
 - untangle::async::invalid_attachment, [30](#)
- is_busy
 - untangle::async::execution< actionT >, [22](#)
- is_running
 - untangle::async::execution< actionT >, [22](#)
 - untangle::async::execution_poll, [29](#)
- namespace untangle: functions, [13](#)
- on_error
 - untangle::async::execution< actionT >, [26](#)
- on_finished
 - untangle::async::execution< actionT >, [26](#)
- remove
 - untangle::async::execution_poll, [28](#)
- results
 - untangle::async::execution< actionT >, [25](#)
- run
 - untangle::async::execution< actionT >, [22](#)
- start
 - untangle::async::execution< actionT >, [23](#)
- stop
 - untangle::async::execution< actionT >, [23](#)
- untangle, [15](#)
- untangle::async::attachment, [17](#)
- untangle::async::execution< actionT >, [17](#)
 - ~execution, [20](#)
 - action_execute, [26](#)
 - action_is_running, [26](#)
 - action_stop, [26](#)
 - add_action, [23](#)
 - attach, [24](#)
 - bind_action_and_function, [21](#)
 - bind_action_and_method, [20](#)
 - create_instance, [20](#)
 - detach, [25](#)
 - execution, [19](#)
 - is_busy, [22](#)
 - is_running, [22](#)
 - on_error, [26](#)
 - on_finished, [26](#)
 - results, [25](#)
 - run, [22](#)
 - start, [23](#)
 - stop, [23](#)
- untangle::async::execution_poll, [27](#)
 - ~execution_poll, [28](#)
 - add, [28](#)
 - is_running, [29](#)
 - remove, [28](#)
 - withdraw, [29](#)
- untangle::async::invalid_attachment, [29](#)
 - invalid_attachment, [30](#)
 - what, [30](#)
- what
 - untangle::async::invalid_attachment, [30](#)
- withdraw
 - untangle::async::execution_poll, [29](#)